SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – ARCHITECTURE / MODELING (DEVSECOPS METHODOLOGY)

Course Code:	CSA5803	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	Develop Complete DevSecOps Architecture, Methodology Documentation and GitHub Submission.		
Student Name:	V.Lokesh kumar	**Reg. No.:**	192521170
Section / Batch:	B.TECH – CSE	**Date:**	22/08/2026

Instructions to Students

- Develop a complete and original DevSecOps architecture showing the end-to-end secure software delivery workflow.
- Include development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring and feedback stages.
- Document the methodology clearly, including security controls, tools, workflow relationships and key design decisions.
- Use clear labels, consistent symbols, proper alignment and professionally formatted architecture diagrams.
- Submit the editable architecture source files, methodology documentation and an accessible GitHub repository containing all required artifacts.

Question Type	Focus Area	Questions	Marks
ARCHITECTURE / MODELING	DevSecOps Methodology	Q1	Q1 – 100
GRAND TOTAL:			**100 Marks**

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Complete DevSecOps Architecture Design	20%	Develops a comprehensive and technically accurate DevSecOps architecture integrating development, security, CI/CD, infrastructure and operations components	Develops a clear DevSecOps architecture containing most required components with only minor technical or integration gaps	Develops a basic DevSecOps architecture with limited integration or several missing components	Provides an incomplete or technically inaccurate DevSecOps architecture with weak component integration
Security Integration Across DevSecOps Lifecycle	30%	Effectively integrates security controls and automated security testing across planning, coding, build, test, deployment and operations stages	Integrates appropriate security controls across most DevSecOps stages with only minor omissions	Shows basic security integration but several lifecycle stages or security controls are missing	Shows limited or incorrect security integration across the DevSecOps lifecycle
Methodology and Workflow Documentation	20%	Clearly documents the complete DevSecOps methodology, workflow, tools, responsibilities and security checkpoints with strong technical justification	Documents the DevSecOps methodology and workflow clearly with minor gaps in explanation or justification	Provides basic methodology documentation with limited workflow details or technical explanation	Provides incomplete or unclear methodology documentation with weak technical reasoning
GitHub Repository and Submission Quality	20%	Submits a complete, well-organized GitHub repository containing architecture artifacts, documentation and required files with clear structure and version history	Submits a properly organized GitHub repository containing most required artifacts with minor structural or documentation issues	Submits a partially organized repository with missing artifacts, limited documentation or inconsistent file structure	GitHub submission is incomplete, poorly organized or missing essential architecture and methodology artifacts
Technical Clarity and Professional Presentation	10%	Presents the DevSecOps architecture, methodology and repository professionally with clear relationships, consistent terminology and strong technical clarity	Presents the work clearly and professionally with only minor clarity or formatting issues	Presents the work with basic clarity but noticeable inconsistencies or limited technical explanation	Presents the work poorly with unclear relationships, inconsistent terminology or insufficient technical explanation

Rubrics:

Criteria	Weightage	Q5 (10)	Total Marks (50)
Complete DevSecOps Architecture Design	25%		
Security Integration Across DevSecOps Lifecycle	30%		
Methodology and Workflow Documentation	20%		
GitHub Repository and Submission Quality	15%		
Technical Clarity and Professional Presentation	10%		

Total per Question	**100 Marks**		

100

PROJECT REPORT

CI/CD PIPELINE AUTOMATION WITH INTEGRATED SAST, DAST, AND SCA

Submitted in Partial Fulfillment of the Requirements for the Course

CSA5803 – DEVSECOPS ENGINEERING AND APPLICATION SECURITY

Presented By
M. Gokul – 192565067

Guided By
Dr. R. Pitchandi
Associate Professor
Department of Applied Machine Learning
SIMATS Engineering

ABSTRACT

Modern software development organizations require applications to be developed, tested, and deployed at a faster rate. Continuous Integration and Continuous Deployment or Continuous Delivery pipelines, commonly known as CI/CD pipelines, play an important role in achieving fast and automated software delivery. These pipelines help developers automate important tasks such as source code integration, application building, testing, and deployment. However, traditional CI/CD pipelines mainly focus on speed, automation, and delivery efficiency. In many cases, security testing is not sufficiently integrated throughout the entire software development lifecycle.

The increasing number of cyber threats and software vulnerabilities has created a need for stronger security integration within modern software development processes. Security vulnerabilities can exist in application source code, third-party dependencies, application configurations, APIs, authentication mechanisms, and runtime environments. If such vulnerabilities are identified only after deployment, they can create security risks, increase remediation costs, and affect the reliability of the software system.

This project presents a DevSecOps-oriented CI/CD pipeline automation framework integrated with Static Application Security Testing, Dynamic Application Security Testing, and Software Composition Analysis. Static Application Security Testing is used to analyze application source code and identify possible coding vulnerabilities. Software Composition Analysis is used to identify vulnerable third-party libraries and open-source dependencies. Dynamic Application Security Testing is used to analyze the application while it is running and identify runtime security weaknesses.

The proposed system integrates security testing into different stages of the CI/CD pipeline. When developers commit code changes, the automated pipeline can perform dependency analysis, source code security analysis, application testing, deployment to a testing environment, runtime security testing, and security validation. This approach supports continuous security testing and helps identify security issues at different stages of software development.

The results presented in the project indicate that the integration of SAST, SCA, and DAST improves the overall security and efficiency of the software delivery process. Early vulnerability detection helps reduce security risks, dependency analysis helps maintain secure software components, and runtime testing helps identify weaknesses that may not be detected through static analysis. Therefore, the proposed system provides a practical and scalable DevSecOps approach for improving secure and reliable software deployment.

Keywords: DevSecOps, CI/CD Pipeline, SAST, DAST, SCA, Security Automation, Vulnerability Detection, Secure Deployment, Continuous Integration, Continuous Delivery.

---

1. INTRODUCTION

Software development has evolved from traditional manual development and deployment methods to highly automated software delivery processes. Modern organizations are required to develop applications quickly while maintaining quality, reliability, and security. Continuous Integration and Continuous Deployment have become important practices for achieving these requirements.

Continuous Integration allows developers to frequently integrate code changes into a shared repository. Automated processes can then build and test the application. Continuous Deployment or Continuous Delivery helps automate the process of delivering the application to the next environment. Together, these practices form a CI/CD pipeline.

Traditional CI/CD pipelines provide several advantages, including:

- Faster software development.
- Frequent code integration.
- Automated application builds.
- Automated software testing.
- Faster delivery cycles.
- Reduced manual deployment work.
- Improved consistency.
- Better collaboration between developers and operations teams.

However, traditional CI/CD pipelines may focus mainly on software functionality, automation, and deployment speed. Security testing may sometimes be performed separately or at the end of the development lifecycle. This approach can lead to vulnerabilities being identified late in the process.

Modern software applications can contain security vulnerabilities in many areas. For example, vulnerabilities can exist in source code, third-party libraries, APIs, authentication systems, application configurations, and runtime environments. Therefore, security must be integrated throughout the software development lifecycle.

DevSecOps is an approach that combines Development, Security, and Operations. The main objective of DevSecOps is to make security a continuous and shared responsibility. Security testing is integrated into the software development pipeline rather than being treated as a separate process.

The proposed project focuses on CI/CD pipeline automation with the integration of three important security testing mechanisms:

1. Static Application Security Testing.
2. Software Composition Analysis.
3. Dynamic Application Security Testing.

SAST helps identify security vulnerabilities in source code. SCA helps identify vulnerabilities in third-party libraries and open-source dependencies. DAST helps identify vulnerabilities in a running application. The combination of these security mechanisms provides a more complete security analysis. The project demonstrates how security automation can be integrated into the CI/CD pipeline to improve software quality, reliability, and deployment security.

---

2. LITERATURE SURVEY

DevSecOps has become an important approach for integrating security practices into modern software development. Several studies and research works have focused on the integration of security testing within Continuous Integration and Continuous Deployment environments.

Behl discussed the concept of integrating security practices into DevOps environments. The study emphasized the importance of detecting security vulnerabilities early and continuously monitoring applications throughout the software development lifecycle.

Research on security practices in Continuous Integration environments has highlighted the importance of

automated static code analysis. Static analysis can help developers identify vulnerabilities during the coding stage before the application is deployed.
Automated security testing has also been identified as an effective method for improving vulnerability detection and reducing manual security effort. Automated tools can perform security analysis continuously whenever code changes are introduced.
Dynamic Application Security Testing has been studied as a technique for identifying runtime vulnerabilities and application misconfigurations. DAST can test the application while it is running and analyze different inputs, responses, APIs, and application behaviors.
Software Composition Analysis has become increasingly important because modern software applications depend heavily on open-source libraries and third-party components. Vulnerabilities in these dependencies can affect the security of the entire application.
Recent DevSecOps research has demonstrated the importance of integrating automated security testing into CI/CD pipelines. However, some existing systems use security tools independently rather than integrating SAST, DAST, and SCA into a unified and automated pipeline.
The literature indicates that continuous and automated security testing is essential for secure software development. The proposed project combines multiple security mechanisms to provide security coverage across different stages of the software lifecycle.

---

3. RESEARCH GAPS

Although many security tools and DevSecOps approaches are available, several gaps still exist in the effective implementation of secure CI/CD pipelines.

3.1 High False Positive Rates

Security tools may sometimes identify issues that are not actual vulnerabilities. These false positive results can increase the workload of developers and security teams.

3.2 Reduced Developer Trust

When security tools produce a large number of false positives, developers may begin to ignore security warnings. This can reduce the effectiveness of automated security testing.

3.3 Lack of End-to-End Automation

Some CI/CD systems integrate only selected security tools. A complete automated process covering source code, dependencies, runtime applications, and security validation may not always be available.

3.4 Limited Real-Time Vulnerability Prioritization

Not all vulnerabilities have the same level of risk. There is a need for better mechanisms to prioritize vulnerabilities based on their severity and potential impact.

3.5 Cloud-Native Security Challenges

Modern applications are increasingly developed using cloud services, containers, and microservices. These environments require additional security integration.

3.6 Fragmented Security Processes

SAST, DAST, and SCA may be implemented as separate activities. This can make the security process more difficult to manage.

3.7 Third-Party Dependency Risks

Applications increasingly use open-source and external libraries. Vulnerabilities in these components can introduce additional security risks.

3.8 Need for Unified Security Frameworks

There is a need for unified frameworks that integrate multiple security testing mechanisms into a single automated pipeline.
The proposed system addresses these gaps by integrating SAST, SCA, and DAST into a DevSecOps-based CI/CD workflow.

4. PROBLEM STATEMENT

Modern software development depends heavily on CI/CD pipelines to provide faster and continuous application delivery. These pipelines automate software building, testing, and deployment. Although automation improves speed and efficiency, security testing may not always be integrated throughout every stage of the pipeline.

The absence of continuous security testing increases the possibility of vulnerabilities being introduced into production environments. If security vulnerabilities are discovered late, the process of fixing them can require additional time and effort.

Security tools such as SAST, DAST, and SCA are often used independently. This can result in fragmented security processes. Developers may need to use different tools separately, analyze multiple reports, and manually coordinate security findings.

In addition, modern applications increasingly depend on third-party libraries, open-source components, APIs, cloud services, and automated deployments. These technologies create additional security risks and increase the attack surface.

Therefore, there is a need for an automated and integrated security framework that continuously analyzes software security throughout the CI/CD pipeline.

The proposed system solves this problem by integrating SAST, SCA, and DAST into a unified automated DevSecOps workflow.

---

5. OBJECTIVES

The main objective of the project is to design and develop an automated CI/CD pipeline integrated with multiple security testing mechanisms.

The specific objectives are:

1. To design an automated CI/CD pipeline for secure software development.
2. To integrate SAST into the development stage.
3. To integrate SCA for dependency analysis.
4. To integrate DAST for runtime security testing.
5. To detect security vulnerabilities early in the software lifecycle.
6. To identify vulnerable third-party libraries.
7. To detect runtime vulnerabilities.
8. To reduce manual security testing effort.
9. To reduce human errors through automation.
10. To improve software quality.
11. To improve deployment reliability.
12. To improve application security.
13. To support continuous security validation.
14. To reduce the risk of deploying vulnerable applications.
15. To demonstrate a practical DevSecOps workflow.

---

6. PROPOSED SYSTEM

The proposed system follows a DevSecOps-oriented approach where security is integrated into the CI/CD pipeline.

The pipeline begins when a developer commits source code changes to the repository. The CI/CD system automatically detects the changes and triggers the pipeline.

The proposed workflow includes the following major stages:

Code Commit
↓
Pipeline Trigger
↓
SAST Analysis
↓
SCA Dependency Analysis
↓
Build Process
↓
Functional Testing
↓
Deployment to Testing Environment
↓
DAST Analysis
↓
Security Validation
↓
Security Gate
↓
Secure Deployment

The proposed system provides security testing at different levels. SAST focuses on the source code, SCA focuses on dependencies, and DAST focuses on the running application.
The system provides a multi-layered security approach.

---

7. SYSTEM ARCHITECTURE

The system architecture consists of several connected components.

7.1 Developer

The developer writes and modifies the application source code.

7.2 Source Code Repository

The repository stores application source code and tracks changes made by developers.

7.3 CI/CD Pipeline

The CI/CD pipeline automates software building, testing, security analysis, and deployment.

7.4 SAST Module

The SAST module analyzes the application source code and identifies potential security vulnerabilities.

7.5 SCA Module

The SCA module analyzes third-party dependencies and open-source libraries.

7.6 Build and Testing Module

This module builds the application and performs required testing.

7.7 Testing Environment

The application is deployed to a controlled environment for runtime testing.

7.8 DAST Module

The DAST module analyzes the running application.

7.9 Security Gate

The security gate evaluates the results of all security tests.

7.10 Deployment Environment

After successful security validation, the application proceeds to deployment.
The overall architecture creates a continuous flow between development, testing, security, and

deployment.

---

8. METHODOLOGY

The project methodology follows a step-by-step automated security workflow.

Step 1: Application Development
Developers write application source code and make changes based on software requirements.

Step 2: Code Commit
The developer commits or pushes the modified code to the repository.

Step 3: Automated Pipeline Trigger
The code change automatically triggers the CI/CD pipeline.

Step 4: Static Security Analysis
SAST analyzes the source code without executing the application.

Step 5: Dependency Analysis
SCA identifies third-party libraries and checks for known security vulnerabilities.

Step 6: Application Build
The application is built automatically.

Step 7: Functional Testing
The application is tested to verify that the expected functionality works correctly.

Step 8: Deployment to Test Environment
The application is deployed to a testing environment.

Step 9: Dynamic Security Analysis
DAST analyzes the running application for security weaknesses.

Step 10: Security Validation
The results from different security tools are analyzed.

Step 11: Security Gate Decision
The system determines whether the application meets the required security conditions.

Step 12: Secure Deployment
After successful validation, the application can proceed to deployment.

This methodology supports continuous integration, continuous testing, continuous security, and automated deployment.

---

9. MODULE 1 – STATIC APPLICATION SECURITY TESTING

Static Application Security Testing analyzes application source code without executing the application. SAST is performed at an early stage of the development lifecycle. It helps identify potential security vulnerabilities before the application is built and deployed.

The main functions of SAST include:

- Analyzing source code.
- Detecting insecure coding practices.
- Identifying insecure functions.
- Detecting logic flaws.
- Identifying SQL injection vulnerabilities.
- Detecting Cross-Site Scripting issues.
- Identifying buffer overflow risks.
- Detecting potential security weaknesses.

The major advantage of SAST is early vulnerability detection. Developers can identify and fix issues during the development stage.

SAST supports the shift-left security approach, where security testing is performed earlier in the software development lifecycle.

---

10. MODULE 2 – SOFTWARE COMPOSITION ANALYSIS

Software Composition Analysis focuses on third-party libraries, dependencies, and open-source components used by an application.

Modern applications frequently use external packages and libraries. These components can provide useful functionality but may also contain known security vulnerabilities.

The main functions of SCA include:

- Identifying third-party libraries.
- Detecting open-source components.
- Identifying outdated dependencies.
- Detecting known vulnerabilities.
- Monitoring dependency versions.
- Supporting license compliance.
- Providing security alerts.
- Helping maintain a secure dependency ecosystem.

SCA is important because a secure application can still become vulnerable when it uses an insecure third-party component.

---

11. MODULE 3 – DYNAMIC APPLICATION SECURITY TESTING

Dynamic Application Security Testing analyzes an application while it is running.

Unlike SAST, DAST does not focus directly on the source code. Instead, it interacts with the running application and analyzes its behavior.

The major functions of DAST include:

- Testing running applications.
- Detecting runtime vulnerabilities.
- Analyzing user inputs.
- Analyzing application responses.
- Testing APIs.
- Identifying SQL injection issues.
- Detecting Cross-Site Scripting vulnerabilities.
- Detecting authentication-related security issues.
- Identifying application misconfigurations.

DAST is performed during the testing or deployment stage of the CI/CD pipeline.

DAST provides an additional security layer because some vulnerabilities may only become visible when the application is running.

---

12. MODULE 4 – SECURITY GATE

The security gate is an important decision point in the proposed pipeline.

The results from SAST, SCA, and DAST are evaluated before deployment.

The security gate can:

- Check the security results.
- Identify serious vulnerabilities.
- Validate the security status.

- Prevent insecure software from progressing.
- Support secure deployment decisions.

The security gate ensures that security is considered as part of the automated deployment process.

---

13. DEVSECOPS INTEGRATION

DevSecOps integrates Development, Security, and Operations.
The proposed system follows the following DevSecOps principles:

Shift-Left Security
Security testing is performed earlier in the development lifecycle.

Continuous Security
Security analysis is performed continuously whenever code changes occur.

Security Automation
Security testing is integrated into automated pipeline stages.

Shared Responsibility
Developers, security teams, and operations teams share responsibility for application security.

Continuous Monitoring
Security issues can be monitored throughout the software lifecycle.
The proposed CI/CD pipeline demonstrates how security can be integrated into automation rather than being treated as a separate process.

---

14. RESULTS AND DISCUSSION

The proposed system integrates SAST, SCA, and DAST at different stages of the CI/CD pipeline.
The results show that different tools provide different types of security information.
SAST helps detect vulnerabilities in application source code. Early detection allows developers to correct potential security problems before the application proceeds further.
SCA helps identify vulnerable third-party dependencies. This is important because modern applications depend heavily on open-source components.
DAST helps identify runtime vulnerabilities in the running application. This provides security information that may not be available through static analysis.
The combined use of these tools provides better security coverage.
The project results indicate:

- Security vulnerabilities can be detected at different stages.
- Early detection reduces security risks.
- Dependency vulnerabilities can be identified.
- Runtime vulnerabilities can be analyzed.
- Security testing becomes more automated.
- Manual effort is reduced.
- The software delivery process becomes more secure.
- Overall pipeline efficiency is improved.

The integration of security testing into the CI/CD pipeline helps organizations identify issues before production deployment.

---

15. ADVANTAGES OF THE PROPOSED SYSTEM

The proposed system provides several advantages.

Security Advantages

- Early detection of vulnerabilities.

- Multiple layers of security testing.
- Improved dependency security.
- Runtime security testing.
- Continuous vulnerability identification.

Development Advantages

- Reduced manual security effort.
- Faster identification of security issues.
- Better developer awareness.
- Improved code quality.
- Continuous automated feedback.

Operational Advantages

- Improved deployment reliability.
- Automated security validation.
- Consistent testing process.
- Reduced human errors.
- Better software delivery process.

Business Advantages

- Reduced security risks.
- Improved application reliability.
- Reduced cost of late vulnerability detection.
- Better support for secure software development.

---

16. LIMITATIONS

The proposed system has some limitations.

1. Security tools may generate false positives.
2. Security analysis may increase pipeline execution time.
3. Large applications may require additional computing resources.
4. Proper security tool configuration is required.
5. Different applications may require different security policies.
6. Complex cloud environments may require additional security controls.
7. Security testing tools may require continuous updates.

These limitations can be addressed through improved tool configuration and future automation techniques.

---

17. FUTURE SCOPE

The proposed system can be further improved in the future.

Artificial Intelligence Integration

AI and Machine Learning can be used for intelligent vulnerability detection.

Vulnerability Prioritization

Future systems can automatically prioritize vulnerabilities based on risk severity.

Real-Time Monitoring

Real-time monitoring can provide continuous visibility into security events.

Predictive Security Analysis

Machine learning models can help predict potential security risks.

False Positive Reduction

Advanced analysis can help reduce false positive results.

Cloud-Native Security

The framework can be extended to cloud-native applications.

Microservices Security

Future systems can provide security testing for multiple independent microservices.

Advanced Automation

Security remediation processes can be further automated.

Autonomous DevSecOps

Future DevSecOps pipelines may automatically detect, prioritize, and manage security issues with minimal manual intervention.

---

18. CONCLUSION

This project presents an automated CI/CD pipeline with integrated Static Application Security Testing, Software Composition Analysis, and Dynamic Application Security Testing.

The project demonstrates the importance of integrating security into the software development lifecycle. Traditional CI/CD pipelines mainly focus on automation and faster delivery, but secure software development requires continuous security validation.

SAST helps detect vulnerabilities in application source code at an early stage. SCA helps identify vulnerabilities in third-party dependencies and open-source components. DAST helps detect security weaknesses in running applications.

The integration of these security mechanisms provides a more complete approach to application security. Each security technique analyzes a different area of the software system.

Automation reduces manual effort and helps perform security testing continuously. Early detection of vulnerabilities can reduce security risks and improve software reliability.

The proposed system demonstrates the practical use of DevSecOps principles in CI/CD environments. Security becomes an integrated part of software development, testing, and deployment.

Overall, the proposed framework provides a scalable and effective approach for secure CI/CD automation. The integration of SAST, SCA, and DAST helps improve security, reliability, and deployment efficiency.

---

19. REFERENCES

1. Behl, A. (2017). *DevSecOps: Integrating Security into DevOps*. IEEE Software.
2. Rahman, M., & Williams, L. (2018). *Security Practices in Continuous Integration Environments*. ACM Digital Library.
3. Sharma, T. (2019). *Automated Security Testing in CI/CD Pipelines*. International Journal of Computer Applications.
4. Kumar, R., & Singh, P. (2020). *Dynamic Application Security Testing in DevOps*. IEEE Access.
5. Zhang, Q. (2021). *Software Composition Analysis for Open Source Risk Management*. Journal of Cybersecurity.
6. Patel, S., & Mehta, D. (2022). *Integrated Security Framework for CI/CD Pipelines*. Springer Journal of Systems Engineering.
7. Reddy, K., & Rao, V. (2023). *Secure CI/CD Pipeline Automation using DevSecOps*. International Conference on DevOps and Cloud Computing.
8. Gupta, N. (2024). *AI-Driven Security Testing in CI/CD Pipelines*. IEEE International Conference on Emerging Technologies.

---

20. ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, **Dr. R. Pitchandi**, for providing valuable guidance, suggestions, encouragement, and continuous support throughout the completion of this project.

We sincerely thank the faculty members of the Department of Applied Machine Learning and SIMATS Engineering for providing academic guidance and a supportive learning environment.
We also extend our gratitude to the technical staff and all those who provided assistance during the completion of this work.
We thank our friends and classmates for their continuous motivation, teamwork, helpful discussions, and support.
Finally, we express our heartfelt gratitude to our families for their encouragement, patience, and unconditional support throughout our academic journey.